Київський національний університет імені Тараса Шевченка

Факультет Інформаційних Технологій

Кафедра мережевих і Інтернет технологій

Лабораторна робота №3

по дисципліні

«Технології створення програмних продуктів»

Шаблони “Adapter”, “Proxy”

Група: МІТм-11

Студент: Соломєнцев Н. А.

Київ 2026

Завдання:

Розробити прототип програми для відправки повідомлень різним способами. Ви використовуєте бібліотеку MessageSender, яка має метод Send(message, recipient), що приймає текст повідомлення та ім’я одержувача й відправляє його за допомогою електронної пошти. Проте ви хочете додати можливість надсилати повідомлення також через SMS і Telegram. Ви знаходите дві інші бібліотеки SMSsender і TelegramSender, які мають методи SendSMS(message, phoneNumber) і SendTelegram(message, username), що вміють відправляти повідомлення через SMS і Telegram відповідно. Але ви не можете змінити код бібліотеки MessageSender, а також не хочете змінювати код вашої програми, що викликає метод Send(message, recipient). Напишіть два класи-адаптери, які будуть реалізовувати інтерфейс бібліотеки MessageSender, але в середині будуть використовувати об’єкти SMSsender і TelegramSender для відправки повідомлень через SMS і Telegram відповідно. Таким чином, ваша програма буде вважати, що вона працює з MessageSender, а насправді вона буде використовувати SMSsender і TelegramSender через адаптери.

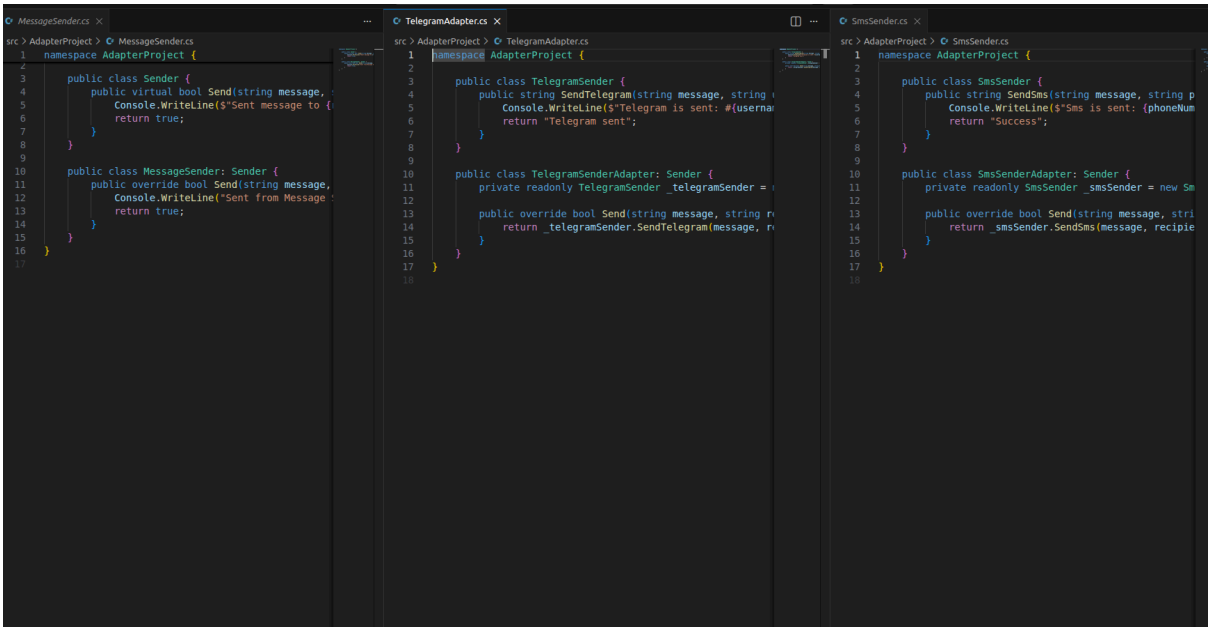

Рис. 1.: Фрагмент коду для реалізації шаблону “Adapter”

Завдання:

Створіть емулятор проксі-сервера для кешування веб-сторінок щоб покращити швидкість завантаження. Коли користувач запитує деяку веб-сторінку, проксі-сервер спочатку перевіряє, чи є ця веб-сторінка в кеші, і якщо так, то надсилає її користувачеві, інакше проксі-сервер отримує її від веб-сервера.

ServerProxy.cs

src > ProxyProject > ServerProxy.cs

```
namespace ProxyProject;

using System;


public class ServerProxy: IServer {
    private uint _cacheTimeToLive;
    private DateTime _cacheLastTimeUpdate;
    private string? _resourceCache = null;
    private string _resourceFilename;

    public ServerProxy(string resourceFilename, uint cacheTtl) {
        _cacheTimeToLive = cacheTtl;
        _resourceFilename = resourceFilename;
        _cacheLastTimeUpdate = DateTime.Now;
    }

    public string GetResource() {
        if (_resourceCache == null || (DateTime.Now - _cacheLastTimeUpdate).TotalSeconds > _cacheTimeToLive) {
            _resourceCache = new Server(_resourceFilename).GetResource();
        }
        return _resourceCache;
    }
}
```

Server.cs

src > ProxyProject > Server.cs

```
namespace ProxyProject;

using System.IO;
using System;


public class Server: IServer {
    private string _filename;

    public Server(string filename) {
        _filename = filename;
    }

    public string GetResource() {
        Thread.Sleep(2000);  // simulate time consuming task
        return File.ReadAllText(_filename);
    }

}
```

IServer.cs

src > ProxyProject > IServer.cs

```
namespace ProxyProject;


public interface IServer {
    public string GetResource();
}
```

Рис. 2.: Фрагмент коду для реалізації шаблону “Proxy”

Висновки:

Шаблон “Adapter” варто використовувати у випадках коли у кодовій базі вже є потрібний функціонал для вирішення певної задачі, але цей функціонал реалізує інтерфейс що є непридатним для вирішення даної задачі. У цьому випадку можна творити окремий новий клас адаптер, що буде реалізовувати потрібний інтерфейс та імплементація буде містити використання вже існуючого коду з кодової бази

Шаблон “Proxy” варто використовувати як оптимізацію програмний викликів що є одночасно як ресурсоємними так і для однакових запитів повертають однаковий результат. Прикладом цього як раз є HTTP сервер що повертає статичні дані